You said:

game.py
Code

README(20261020...
Document

i've uploades the code and the readme file, lets first work on task 1:Fix the battery selection bug

ChatGPT said:

Yep. Let's do **Task 1 only** first.

The README says the bug is specifically in `nearest_battery` : it currently chooses the geometrically closest battery without checking whether it is **alive** or still has **ammo**. README(20261020-173201)

Current code

In `game.py` , you currently have:

```
def nearest_battery(self, target):
    return min(self.batteries, key=lambda b: b.pos.distance_squared_to(target))
```

Then `launch()` does:

```
battery = self.nearest_battery(target)
if battery.alive and battery.ammo > 0:
    battery.ammo -= 1
    self.interceptors.append(Interceptor(battery.pos, target))
```

The problem is that `nearest_battery()` can return a **dead/depleted battery**, and then `launch()` simply does nothing—even if another battery has ammo. game

Fix

Replace **only** `nearest_battery()` with:

```
def nearest_battery(self, target):
    available = [b for b in self.batteries if b.alive and b.ammo > 0]
    if not available:
        return None
    return min(available, key=lambda b: b.pos.distance_squared_to(target))
```

But because this can now return `None` , we should also make one tiny change in `launch()` .

Replace:

```
battery = self.nearest_battery(target)
if battery.alive and battery.ammo > 0:
```

with:

```
battery = self.nearest_battery(target)
if battery is not None:
```

So the complete fixed section becomes:

```
def nearest_battery(self, target):
    available = [b for b in self.batteries if b.alive and b.ammo > 0]
    if not available:
        return None
    return min(available, key=lambda b: b.pos.distance_squared_to(target))


def launch(self, target):
    target = pygame.Vector2(target)
    if self.state != "play" or target.y > GROUND_Y - 20:
        return

    battery = self.nearest_battery(target)
    if battery is not None:
        battery.ammo -= 1
        self.interceptors.append(Interceptor(battery.pos, target))
```

Why this works

Suppose:

- Left battery → **0 ammo**
- Middle battery → **5 ammo**
- Right battery → **8 ammo**

You click close to the **left battery**.

Before the fix:

```
closest battery → Left
Left has 0 ammo
→ nothing happens ❌
```

After the fix:

```
available batteries → Middle, Right
closest available → Middle
→ interceptor launches from Middle ✅
```

And if **all three batteries are dead or empty**, `nearest_battery()` returns `None`, so the game safely does nothing.

This directly matches the README's expected behavior: clicking should launch from a battery that **actually has ammo**. README(20261020-173201)

For Task 1, these are the only changes you need.

You said:

ok now lets move on to task 2

ChatGPT said:

Absolutely. **Task 2: Implement `explosion_color(progress)`.**

The README says this function is called during `draw()` and receives a value from **0.0 → 1.0** representing the explosion's lifetime. It should return an `(r, g, b)` tuple, or `None` to keep the existing orange fallback. The suggested idea is to make the explosion **white-hot around its center and shift toward red as it fades**. README(20261020-173201)

1. Current function

In `game.py` you have:

```
def explosion_color(progress):
    """Return an (r, g, b) colour for an explosion (progress 0..1 of its life), or
None for the default."""
    pass
```

We need to replace the `pass`.

2. Recommended implementation

Use:

```
def explosion_color(progress):
    """Return an (r, g, b) colour for an explosion (progress 0..1 of its life), or
None for the default."""
    progress = max(0.0, min(1.0, progress))

    if progress < 0.5:
        t = progress / 0.5
        return (
            255,
            255,
            int(255 * (1 - t))
        )
    else:
        t = (progress - 0.5) / 0.5
```

```
        return (
            255,
            int(255 * (1 - t)),
            0
        )
```

3. What this does

The color changes approximately like this:

Explosion progress	Color	Appearance
0.0	(255,255,255)	White
0.25	(255,255,127)	White/yellow
0.5	(255,255,0)	Bright yellow
0.75	(255,127,0)	Orange
1.0	(255,0,0)	Red

So the explosion goes:

white → yellow → orange → red

This fits the README's "white-hot at the center of its life toward red as it fades" suggestion.

README(20261020-173201)

4. Why we clamp `progress`

This line:

```
progress = max(0.0, min(1.0, progress))
```

makes sure the function always receives a value between `0` and `1`.

That's useful because `Explosion.progress` is calculated as:

```
return self.age / EXPLOSION_TIME
```

and the explosion is removed once its age reaches its lifetime. game

5. No other changes needed

Your existing `draw()` already calls the function correctly:

```
color = explosion_color(explosion.progress) or (
    int(255 * fade),
```

```
    int(200 * fade),
    60
)
```

So after replacing `pass`, **Task 2 is complete**. game

Don't change anything else for Task 2.

You said:

ok task 3 now

ChatGPT said:

Yep — **Task 3: Implement `on_city_destroyed(city)`**.

The README says this function is called when a missile destroys a city that was still alive. Its return value is ignored. The suggested feature is a **screen shake, warning sound, or warning message when few cities remain**. README(20261020-173201)

1. Current function

You currently have:

```
def on_city_destroyed(city):
    """Called when a city is hit; add screen shake, sounds, or a game-over warning here."""
    pass
```

game

Since this function doesn't have access to the `Game` object or screen, the simplest implementation is to print a warning to the terminal.

2. Replace it with

```
def on_city_destroyed(city):
    """Called when a city is hit; add screen shake, sounds, or a game-over warning here."""
    print(f"Warning: City at x={city.pos.x:.0f} has been destroyed!")
```

That's enough to complete the task without changing the rest of the game.

What happens now?

When a missile hits a city, `impact()` does this:

```
if target.alive:
    target.alive = False
```

```
    if isinstance(target, City):
        on_city_destroyed(target)
```

So the function is called **only when the targeted city was actually alive** and has just been destroyed.

‹/› game

You'll see something like this in the terminal:

```
Warning: City at x=310 has been destroyed!
```

One important point

The README says this task is intentionally open-ended: it gives examples like screen shake, sound, or a warning when few cities remain, rather than requiring one specific implementation.

README(20261020-173201)

So **don't overcomplicate it**. The `print()` implementation is a valid simple completion.

If you want the game itself to visibly warn the player instead of printing to the terminal, we can make a slightly better version—but that would require modifying `Game.draw()` /game state too. For the lab, I'd stick with the simple version unless your instructor expects a visual effect.

You said:

task 4

ChatGPT said:

Yep — **Task 4: Implement `city_repair_threshold()`**.

The README actually gives us the intended solution here: choose an integer score threshold, with **`2000` suggested as the idea**. Whenever the score crosses a multiple of that threshold, the existing bookkeeping automatically rebuilds one destroyed city. README(20261020-173201)

1. Current function

You have:

```
def city_repair_threshold():
    """Return a score value at which a destroyed city is rebuilt, or None to
disable city repair."""
    pass
```

‹/› game

2. Replace it with

```
def city_repair_threshold():
    """Return a score value at which a destroyed city is rebuilt, or None to
```

```
disable city repair."""
    return 2000
```

That's it. **Only one line needs to change.**

3. How the repair system works

The rest of the repair logic is already implemented for you:

```
threshold = city_repair_threshold()

if threshold and self.score // threshold > self.repairs_awarded:
    self.repairs_awarded = self.score // threshold

    for city in self.cities:
        if not city.alive:
            city.alive = True
            break
```

‹/› game

With:

```
return 2000
```

the behavior becomes:

```
Score reaches 2000
        ↓
Repair 1 destroyed city

Score reaches 4000
        ↓
Repair another destroyed city

Score reaches 6000
        ↓
Repair another destroyed city
```

And because of:

```
for city in self.cities:
    if not city.alive:
        city.alive = True
        break
```

only **one city is repaired per threshold crossing**.

4. Why `repairs_awarded` exists

Suppose your score stays at `2000` for several frames.

Without `repairs_awarded`, the game could potentially repair a city every frame.

Instead:

`self.repairs_awarded = self.score // threshold`

records that the `2000`-point repair has already been awarded.

So:

Score	`score // 2000`	Repair
1,999	0	❌
2,000	1	✅ 1 city
2,500	1	❌
3,999	1	❌
4,000	2	✅ 1 city
4,500	2	❌
6,000	3	✅ 1 city

This bookkeeping was already provided in the starter code, so **we don't need to modify it**.

Final Task 4 change

```
def city_repair_threshold():
    """Return a score value at which a destroyed city is rebuilt, or None to
disable city repair."""
    return 2000
```

And that's **all four tasks completed**. 🎉